

1 Part 8: Dimension Reduction

2 Summary

3 Often data come to us in a **high-dimensional form**, meaning that each ob-
4 servation is actually best thought of as a vector of many numbers.

5 In some situations, it is crucial that lower-dimensional representations of
6 these data are obtained. Here we present some approaches to doing that.

7 First, we motivated why this dimension reduction can be so important.

1 The Curse of Dimensionality

2 Despite the promise of nonparametric approaches, they suffer from a seri-
3 ous drawback: The amount of data required to fit nonparametric density
4 estimates or regressions well increases exponentially with the dimension
5 of the data.

6 This is called the **Curse of Dimensionality**.

- 1 To start, we ask: **What is the “dimension” of data?**
- 2 Standard data come to us in the form of numbers, but a single **observation**
- 3 is often best thought of as a list of numbers.
- 4 For example, a **yield curve** shows how bond rates vary as a function of
- 5 time to maturity. A single yield curve consists of several numbers, usually
- 6 around ten rates. This count of how many numbers it takes to represent a
- 7 yield curve is called its **dimension**. We would say that a single yield curve
- 8 is “ten-dimensional.”
- 9 Another example: Imagine our data consists of separate price time series
- 10 for each of 100 stocks. Each series goes back 20 trading days. We would say
- 11 in this case that we have “100 observations of 20-dimensional time series”.

- 1 One place where the dimension of data often comes up is in the context
- 2 of regression or supervised learning. If there are p predictors being used
- 3 in a model for a response variable, we would say that we are using a “ p -
- 4 dimensional predictor vector.”
- 5 Nonparametric regression is possible when the predictor vector is p -
- 6 dimensional. For instance, if $p = 2$, a neighborhood can be thought of
- 7 as a square centered on the target value. Local linear regression involves
- 8 fitting a plane within this square.
- 9 Reconsider our options sample from the previous Part. The figure on the
- 10 following page shows a two-dimensional local linear regression fit. The
- 11 response is the “error” in the Black-Scholes price (relative to the ask price)
- 12 and the predictors are the time to expiration and the strike price.

- 1 First, read in the data and fit the model:

```
> optionsdata = read.table("optionssample09302017.txt",
+                           header=T, sep=",")
> optionsdata$bserr = optionsdata$bsval - optionsdata$ask
>
> holdlo2d = loess(bserr ~ timetoexpiry + strike,
+                  data=optionsdata, degree=1)
```

- 2 The function `expand.grid()` creates a two-dimensional grid of values
- 3 from two vectors.

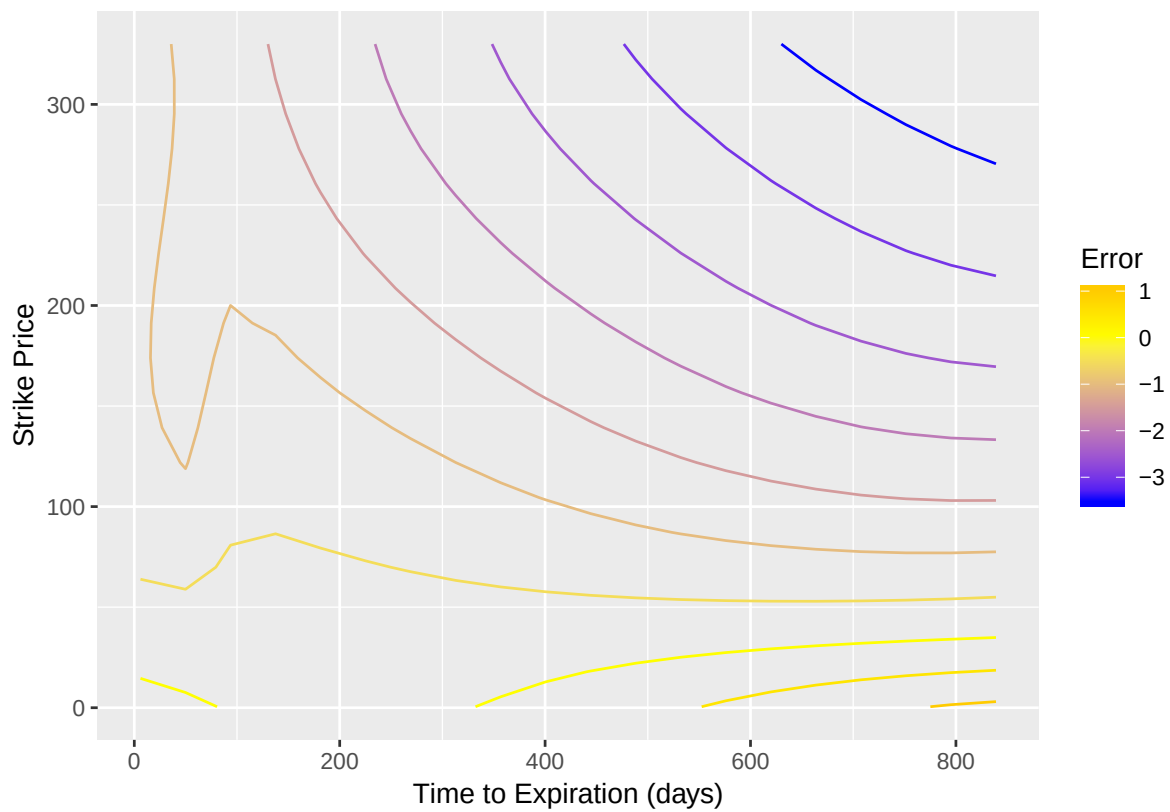
```
> fullgrid = expand.grid(
+   timetoexpiry=
+     seq(min(optionsdata$timetoexpiry),
+         max(optionsdata$timetoexpiry), length=20),
+   strike=seq(min(optionsdata$strike),
+              max(optionsdata$strike), length=20))
```

- 1 Now, make the predictions on this grid:

```
> fits = predict(holdlo2d, newdata=as.matrix(fullgrid))
```

- 2 Finally, create the plot. Note the use of the syntax `color=..level..`
- 3 in order to specify that the colors should reflect the different levels in the
- 4 contour plot.

```
> fitframe = data.frame(cbind(fullgrid, fits))
>
> ggplot(fitframe, aes(x=timetoexpiry, y=strike, z=fits)) +
+   geom_contour(aes(color=..level..)) +
+   scale_color_gradient2(low="blue", high="red",
+                          mid="yellow", midpoint=0) +
+   labs(x="Time to Expiration (days)", y="Strike Price",
+        color="Error")
```



- 1 **Exercise:** The next plot adds on the 1,000 observed predictor pairs used in
- 2 this case. Sketch the neighborhood when the regression function is esti-
- 3 mated at (400, 200). What will have to happen in order for this estimate to
- 4 have sufficiently low variance?

5 _____

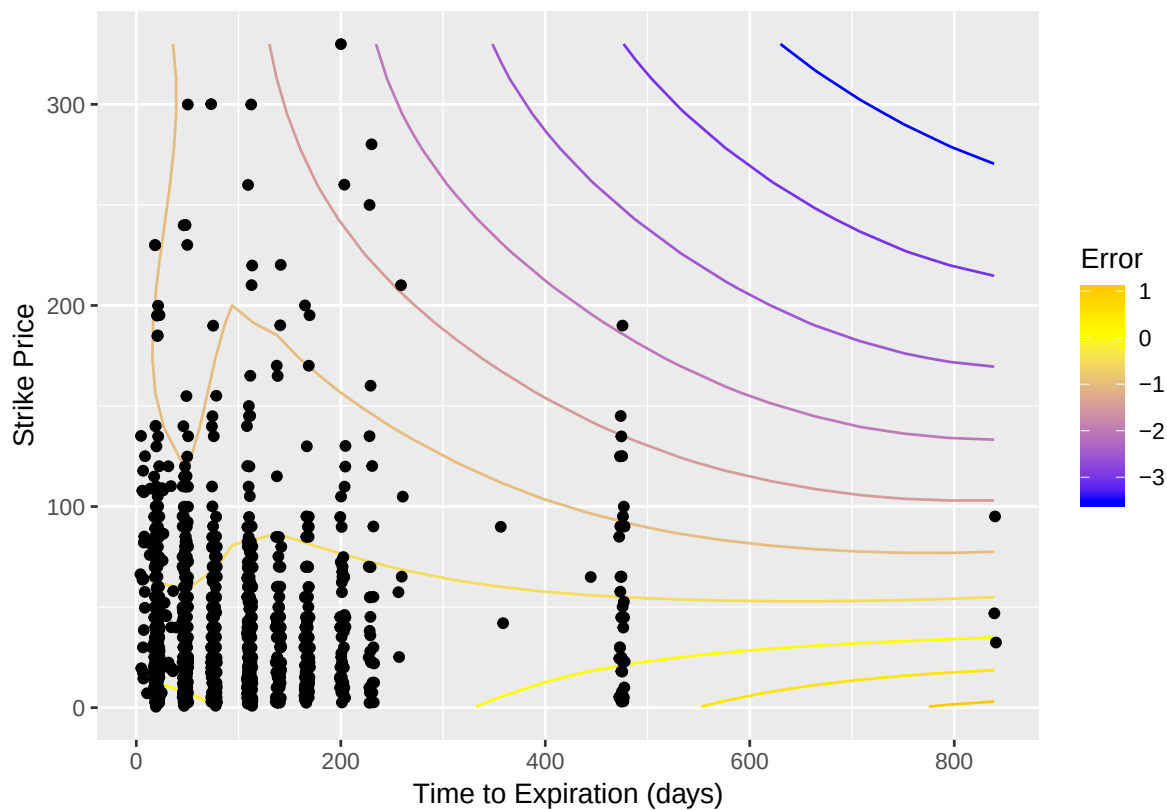
6 _____

7 _____

8 _____

9 _____

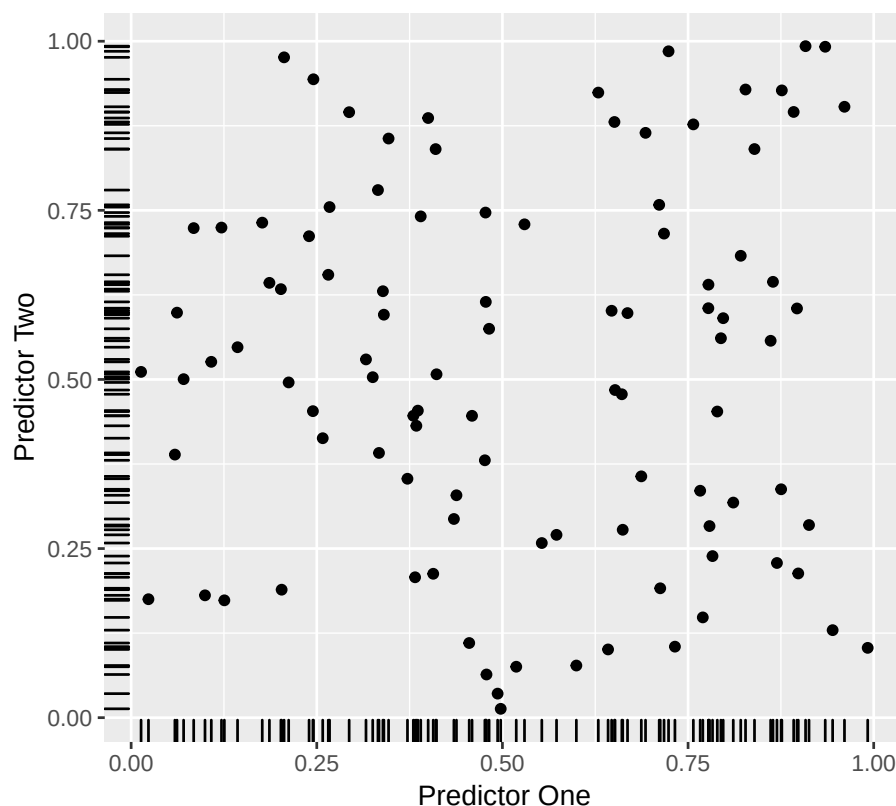
10



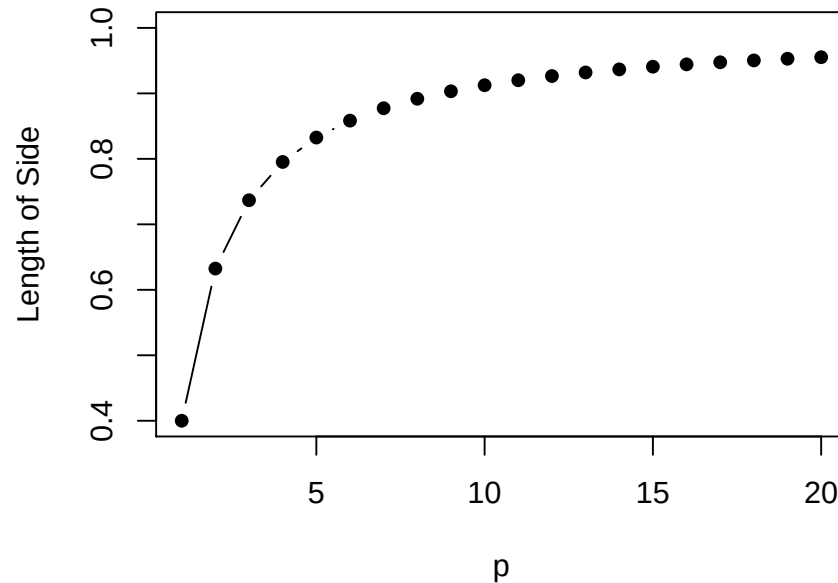
1

- 1 This example shows the fundamental challenge of working with high-
- 2 dimensional predictors: As the dimension increases, the available obser-
- 3 vations become more “spread out.” Neighborhoods must be made larger
- 4 in order to compensate and achieve estimates with acceptable variance.
- 5 But, choosing a large neighborhood takes away a key feature of using non-
- 6 parametric approaches: We want the estimate to be flexible, and not be
- 7 based on too much “smoothing.”

- 1 Consider the plot on the following slide.
- 2 Here, the sample size is 100. Each observation is two-dimensional, i.e.,
- 3 imagine these are the two predictors. These data are simulated from uni-
- 4 form distributions.
- 5 The “marks” shown in each margin depict the **marginal distribution** for
- 6 each predictor. Note that in the margins, the data appear to be quite closely
- 7 spaced. If we were to fit a nonparametric regression using just one predic-
- 8 tor, neighborhoods could be chosen small.
- 9 But, in two dimensions, large gaps appear in the distribution of the obser-
- 10 vations. Neighborhoods will have to be chosen larger in order to capture
- 11 enough data.



- 1 Another way of thinking about it: Under the uniform setup, with a p -
2 dimensional predictor, in order for a neighborhood to include proportion
3 α of the data, the length of the sides of the neighborhood must be $\alpha^{1/p}$. This
4 plot shows the case where $\alpha = 0.4$.



5

- 1 **Exercise:** Suppose that $p = 5$. Under our uniform setup, how long are
2 the sides of a neighborhood that covers 40% of the data? What are the
3 implications of this?

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11

1 **Exercise:** Again assume our uniform predictor distribution. Define that an
2 observation is “close to the boundary” if it is either less than 0.1 or greater
3 than 0.9 for at least one predictor. What proportion of the observations are
4 “close to the boundary?”

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

12 _____

1 The curse of dimensionality motivates us to consider methods which cre-
2 ate **low-dimensional representations** of data.

3 In many cases, data which are p -dimensional in their original form can be
4 compressed to q dimensions, where $q \ll p$, with minimal loss of important
5 information.

6 The aforementioned yield curves provide an important example. The daily
7 yield curve consists of approximately 10 rates, but a yield curve takes a
8 limited class of shapes. Shifts in the yield curve can be described using
9 three numbers in most situations. Hence, a 10-dimensional object can be
10 described compressed to three dimensions.

1 **Web Scraping Example: Obtaining Treasury Yield Curves**

2 **Web scraping** refers to automatically pulling data off of web sites.

3 Here we will illustrate an example of doing this for a data set of interest
4 using the R package `rvest`.

5 Every implementation of web scraping is going to present its own chal-
6 lenges, but the general concepts presented here should be useful.

1 The data we will use in this example consist of daily treasury yield curves,
2 found at the following URL:

3 <https://www.treasury.gov/resource-center/data-chart-center/interest-rates/Pages/TextView.aspx?data=yieldAll>

4 which I have conveniently shortened to <https://goo.gl/j97141>

5 Data on this page go back to the start of 1990.

6 A **yield curve** is simply the relationship between the yield for the bonds of
7 a single type and the time to maturity.

- 1 To get started, install the package `rvest` and load it up. Then use the
2 function `read_html()` to read in the desired web page:

```
> fullyCweb = read_html("https://goo.gl/j97141")
```

- 3 The function `html_nodes()` allows ones to easily extract elements of the
4 web page by specifying **CSS selectors**, which are identifiers that one can
5 use to define formatting for elements of similar types.

- 6 See <https://www.w3schools.com/cssref/tryel.asp> for illustrative exam-
7 ples.

- 1 **Exercise:** Take a moment and study the **source** of the target web page. (All
2 web browsers have an option that allows you to view the source.) Find
3 where in the source the desired data resides.

4 _____

5 _____

6 _____

7 _____

8 _____

9

1 A typical piece of the data table appears as follows:

```
2 <td class="text_view_data">7.83</td><td class="text_view_data">7.89</td>
```

3 In this example, `text_view_data` is a CSS selector. It is a “class,” mean-
4 ing that it is a selector shared across different elements. These are defined
5 using an extra period, e.g., `.class_name` and this is how they should be
6 referenced within `html_nodes()`.

7 So, the following command will extract all of the nodes which include that
8 CSS selector:

```
> tvdnodes = html_nodes(fullyCweb, ".text_view_data")
```

1 You will note that the elements of `tvdnodes` contain extra, undesired in-
2 formation. Fortunately, `rvest` has another function, `html_text()` which
3 strips away the unwanted wrappers.

```
> tableelements = html_text(tvdnodes)
> print(tableelements[1:10])
[1] "01/02/90"          "\n\t\t\tN/A\n\t\t" "7.83"
[4] "7.89"              "7.81"              "7.87"
[7] "7.90"              "7.87"              "7.98"
[10] "7.94"
```

- 1 One clear problem is that the N/A values in the original table are now ap-
- 2 pearing with extra formatting characters.
- 3 The `grep()` function allows for a search over **regular expressions**. This
- 4 is a large topic, but, briefly stated, regular expressions allow for searching
- 5 for strings that meet flexible requirements.
- 6 Here we simply use the following command to return all entries in
- 7 `tableelements` that contain N/A:

```
> grep("N/A", tableelements)
```

- 8 So, the following command will clean things up:

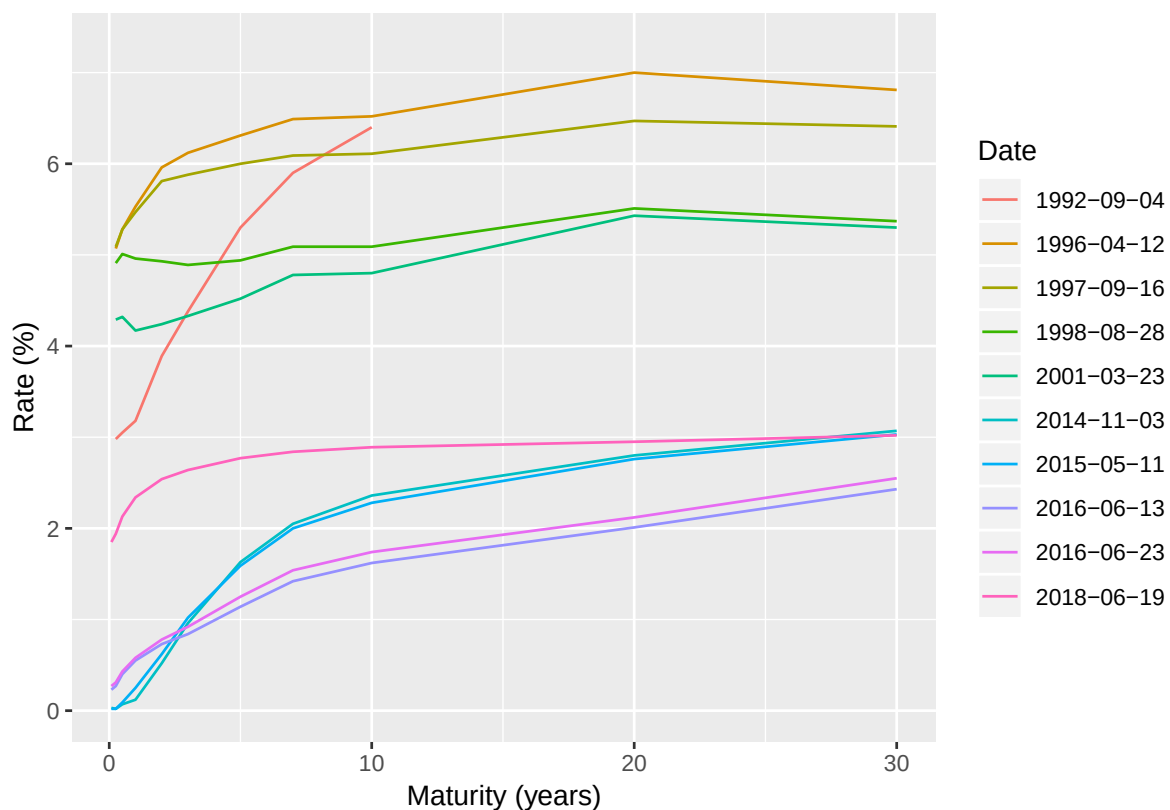
```
> tableelements[grep("N/A", tableelements)] = NA
```

- 1 Now let's get the table into a proper format:

```
> YCdata = matrix(tableelements, ncol=12, byrow=TRUE)
> YCdata = data.frame(YCdata, stringsAsFactors=FALSE)
> names(YCdata) = c("Date", "1mo", "3mo", "6mo", "1yr",
+                  "2yr", "3yr", "5yr", "7yr", "10yr",
+                  "20yr", "30yr")
> YCdata$Date = as.Date(YCdata$Date, format="%m/%d/%y")
> YCdata[, 2:12] = apply(YCdata[, 2:12], 2, as.numeric)
```

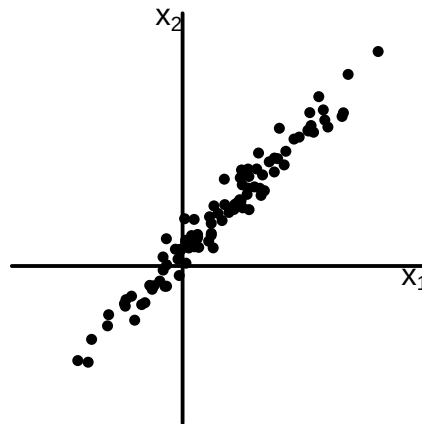
- 1 The package `reshape2` has some nice functions for adjusting data frames
- 2 to get them into a desired form. Here, in order to create a plot we need the
- 3 rates in a “long” form, with one rate per line. The `melt()` function will
- 4 do this:

```
> library(reshape2)
> YCmelt = melt(YCdata, id.vars="Date")
> maturities = c(1/12, 3/12, 6/12, 1, 2, 3, 5, 7, 10, 20, 30)
> YCmelt$maturity =
+   maturities[as.integer(YCmelt$variable)]
> YCmeltsub =
+   YCmelt[YCmelt$Date %in% sample(YCmelt$Date, size=10), ]
> ggplot(YCmeltsub, aes(x=maturity, y=value,
+   color=factor(Date))) +
+   geom_line() +
+   labs(x="Maturity (years)", y="Rate (%)", color="Date")
```



Principal Components Analysis

A standard, classic tool for dimension reduction is **principal components analysis (PCA)**. It has some drawbacks, mostly in that it is best suited to find **linear** structure in data, but in the right situation it can be a very effective tool. First, consider the synthetic data shown in the following figure.



Exercise: Comment on the structure seen in the pair of variables shown in the previous figure. Would you say that the current pair of axes is the most “natural” way to represent the data?

4 _____

5 _____

6 _____

7 _____

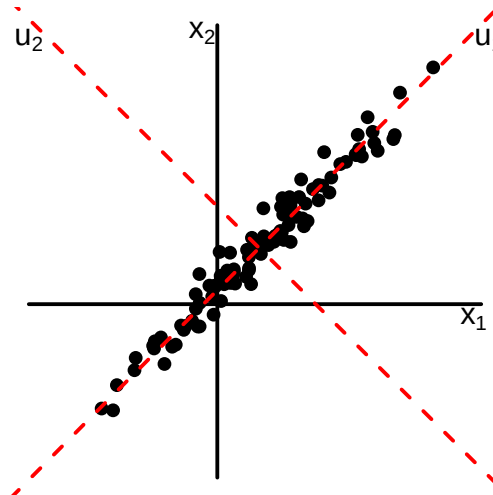
8 _____

9 _____

10 _____

11 _____

- 1 The figure below shows the same data, with a new (u_1, u_2) coordinate sys-
 2 tem shown as the dashed lines.



- 3
 4 These axes seem to be a more natural representation of the data. In partic-
 5 ular, we note that the u_1 axis captures the major source of variability in the
 6 data. These axes are formed from the **principal components** of the data.

1 **Formal Setup of PCA**

- 2 Suppose we have observed n vectors $\mathbf{x}_i$, each of which is of dimension p :

$$3 \quad \mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{ip})^T, \quad i = 1, 2, \dots, n.$$

- 4 For example, $\mathbf{x}_i$ could be a single yield curve, with $p \approx 10$.

- 5 Consider the collection of all possible linear combinations of the original
 6 variables formed by different choices of $\mathbf{u}$:

$$7 \quad v_i = \sum_{j=1}^p u_j (x_{ij} - \bar{x}_j), \quad i = 1, 2, \dots, n$$

- 8 where $\bar{x}_j$ is the mean of the j^{th} variable, subject to

$$9 \quad \sum_{j=1}^p u_j^2 = 1.$$

- 1 Hence, we can think of each choice of $\mathbf{u}$ as being a different **direction** cen-
2 tered on the sample mean.
- 3 The new measurements $v_1, v_2, \dots, v_n$ are the **projections** of the original ob-
4 servations onto this new direction.
- 5 **We now ask:** What choice of $\mathbf{u}$ maximizes the sample variance of the re-
6 sulting numbers $v_1, v_2, \dots, v_n$?



- 1 **The Math:**
- 2 If $\mathbf{A}$ is a symmetric, positive definite matrix then the choice of $\mathbf{u}$ that max-
3 imizes $\mathbf{u}^T \mathbf{A} \mathbf{u}$ subject to $\mathbf{u}^T \mathbf{u} = 1$ is the first eigenvector of $\mathbf{A}$.
- 4 Recall that
- $$V(\mathbf{u}^T \mathbf{x}) = \mathbf{u}^T \mathbf{\Sigma} \mathbf{u}$$
- 5
- 6 where $V(\mathbf{x}) = \mathbf{\Sigma}$.
- 7 We approximate $\mathbf{\Sigma}$ using the sample covariance matrix $\widehat{\mathbf{\Sigma}}$.
- 8 The leading eigenvector of $\widehat{\mathbf{\Sigma}}$ is the first principal component.
- 9 The additional principal components $\mathbf{u}_2, \mathbf{u}_3, \dots, \mathbf{u}_p$ are found by finding
10 successive eigenvectors of $\widehat{\mathbf{\Sigma}}$.

1 **Some comments:**

2 A new coordinate system is being constructed in which the p -dimensional
3 data are represented. The center of this coordinate system is the sample
4 mean of the data, but the axes are the principal components $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_p$.

5 When expressed in this new coordinate system, each of the dimensions
6 have sample correlation of zero.

7 The amount of “variance explained” by each principal component is less
8 than each of the previous principal components.

1 In many situations, we will choose some cutoff $q \ll p$ such that we will
2 only retain the first q axes in our new coordinate system. This will be
3 because these first q components “capture” most of the variability in the
4 data.

5 For instance, in the toy example above, most of the variability in the data
6 is “captured” by the first axis $\mathbf{u}_1$.

7 In typical applications p will be very large, and a low-dimensional repre-
8 sentation will be of great value.

1 Back to the Yield Curves

2 Our previous inspection of some yield curves suggested that these curves
3 share a common shape, and that it does not require ten numbers to de-
4 scribe changes in this shape from day to day.

5 This is an ideal situation to consider dimension reduction. The simplicity
6 of the shapes of the curves suggests that PCA is worth trying.

7 For this analysis we will use the yield curves from 2010 to the present. We
8 need to extract the relevant rates to run the PCA. Note that there are 11
9 rates over this time period.

```
> YCrates = YCdata[YCdata$Date > "2010-01-01", -1]
```

1 There is one row that is filled with zeros:

```
> YCrates = YCrates[-which(apply(YCrates, 1, sum) == 0), ]
```

2 We want to characterize shifts in the yield curves, so we transform into the
3 day-to-day change in the rates:

```
> YCshifts = apply(YCrates, 2, diff)
```

4 The function `princomp()` runs the PCA. The `subset` argument is used
5 to skip over the rows with NA.

```
> pcaout = princomp(YCshifts,  
+                   subset=complete.cases(YCshifts))
```

- 1 The following output is useful:

```
> summary(pcaout)
Importance of components:
```

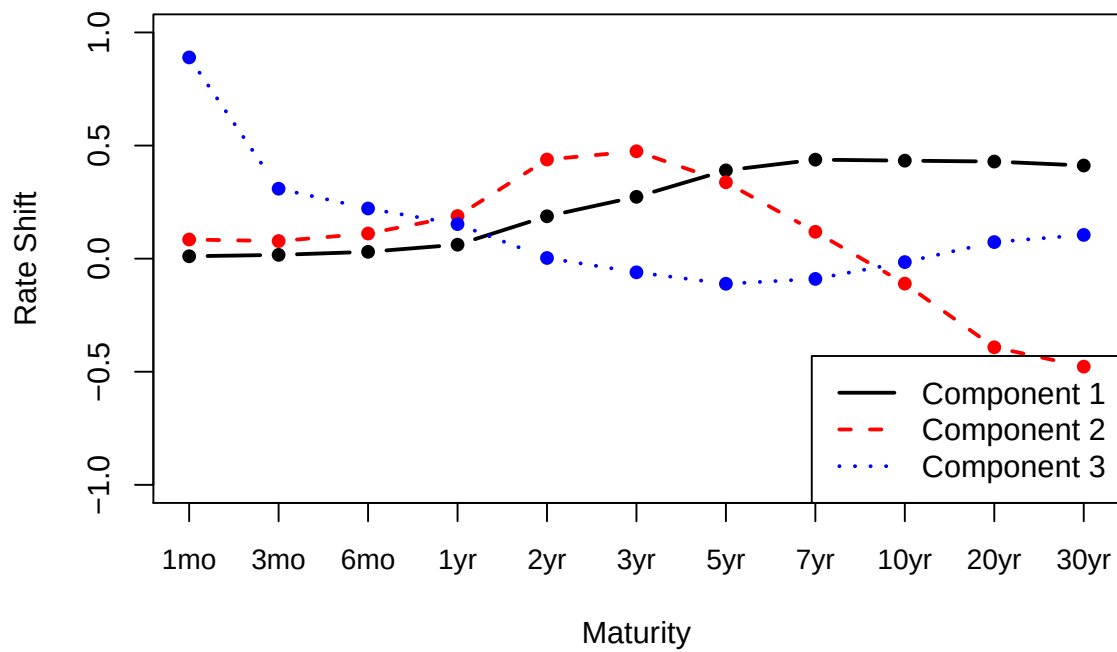
	Comp.1	Comp.2	Comp.3	Comp.4
Standard deviation	0.1132046	0.03442868	0.02476141	0.01669611
Proportion of Variance	0.8253307	0.07633800	0.03948671	0.01795273
Cumulative Proportion	0.8253307	0.90166871	0.94115543	0.95910815

	Comp.5	Comp.6	Comp.7	Comp.8
Standard deviation	0.01431992	0.011679546	0.009897267	0.009073786
Proportion of Variance	0.01320631	0.008785207	0.006308566	0.005302457
Cumulative Proportion	0.97231446	0.981099670	0.987408236	0.992710693

	Comp.9	Comp.10	Comp.11
Standard deviation	0.006819678	0.005906935	0.005637764
Proportion of Variance	0.002995214	0.002247111	0.002046982
Cumulative Proportion	0.995705907	0.997953018	1.000000000

- 2 This shows the proportion of variance explained by each component.

- 1 The first three components account for approximately 95% of the variability
- 2 in the shifts in the yield curves.
- 3 The actual components (directions) are called the **loadings**. Look at
- 4 `pcaout$loadings` to see these vectors. The following plot shows the
- 5 first three components/loadings.
- 6 The interpretation is as follows: The day-to-day change in the yield curves
- 7 can mostly be modelled as a linear combination of the three curves shown
- 8 in this figure. The weight placed on each component (called the **score**) will
- 9 change from day to day.



1

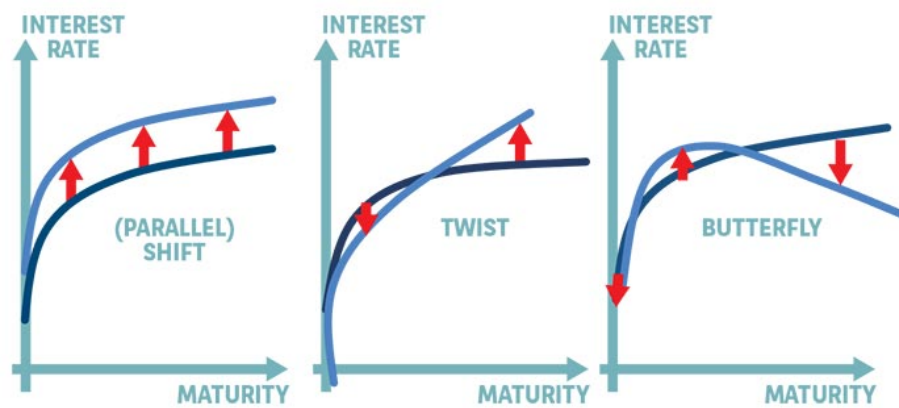


Figure 1: Figure from advisoranalyst.com

- 1 The scores give a **representation** of the shifts in yield curves. In fact, this is
- 2 related to a common way to characterize how the yield curve has changed.
- 3 The first three components are commonly given the names “parallel shift,”
- 4 “twist,” and “butterfly.”

- 1 The `predict()` function, when applied to the output of `princomp()`
- 2 will return the position of yield curve shift in this new representation. Note
- 3 that R is a little picky on the format. The new shift must come in as a data
- 4 frame with column names the same as used in the data.

```
> newshift = matrix(c(0.00, 0.00, 0.01, 0.01, 0.01, 0.07, 0.05, 0.05,
+                    0.03, 0.03, 0.04), nrow = 1)
> newshift = data.frame(newshift)
> names(newshift) = colnames(YCshifts)
> predict(pcaout, newshift)
```

	Comp.1	Comp.2	Comp.3	Comp.4	Comp.5	Comp.6
[1,]	0.1060112	0.02753681	-0.005937651	0.003564653	-0.00352043	0.003759287

	Comp.7	Comp.8	Comp.9	Comp.10	Comp.11
[1,]	-0.005728047	-0.03691347	-0.0005428808	0.009590538	0.006370914

- 1 **Scaling Variables for PCA**
- 2 In cases where the variables being incorporated into PCA are on different
- 3 scales, it is crucial that the variables be standardized prior to the analysis.
- 4 It is customary to scale variables so that each has sample mean of zero, and
- 5 sample variance of one.
- 6 This is equivalent to finding the eigenvectors of the correlation matrix in-
- 7 stead of the covariance matrix. In fact, with function `princomp()`, one
- 8 can simply use argument `cor = T` to achieve this.

1 Nonlinear Dimension Reduction

2 Consider the figure on the following slide. In this case, an individual
3 “observation” is an image of a face. These faces are mapped into a two-
4 dimensional space. The faces along the bottom show how the faces evolve
5 along the line in the upper right of the figure.

6 This illustrates the use of **nonlinear dimension reduction** on high-
7 dimensional data. PCA would not have been a good choice in this
8 case because individual images cannot be approximated by a linear
9 combination of a “basis” faces.

1 **Exercise:** What is the utility of the representation shown in the previous
2 figure?

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10

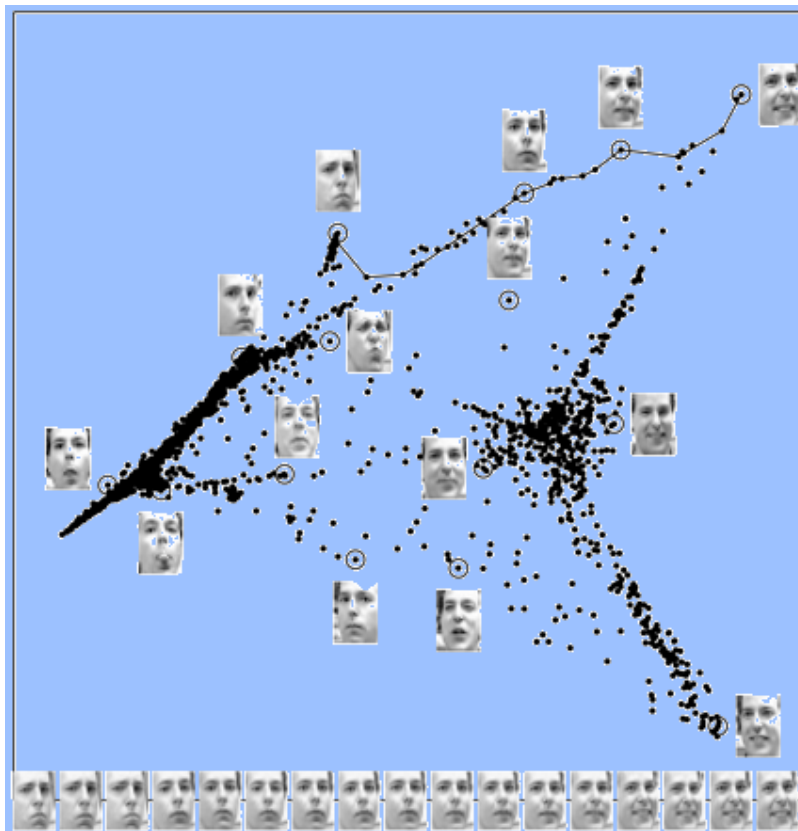
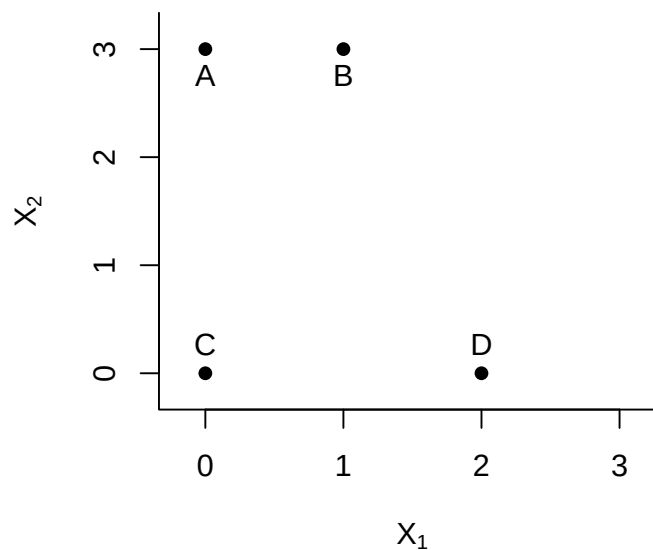


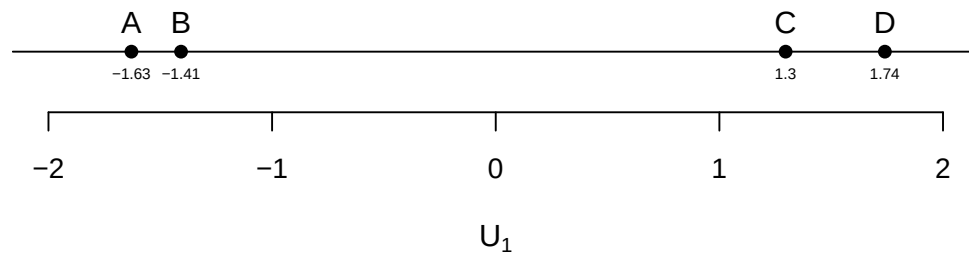
Figure 2: From Roweis and Saul (2000), *Science*

1 Multidimensional Scaling

- 2 Consider a synthetic case in which we have four points in two-dimensions,
- 3 labelled as in the following figure.



- 1 Imagine calculating all of the pairwise distances between the four points
- 2 shown in the above figure.
- 3 Then ask: **What one-dimensional representation of the data preserves**
- 4 **those distances to the greatest extent possible?**
- 5 The answer is addressed by **multidimensional scaling (MDS)**.
- 6 The result, using the classic approach, is shown in the figure below.



- 1 **Exercise:** What is the practical motivation behind trying to find a low-
- 2 dimensional representation that preserves distances?

- 3 _____
- 4 _____
- 5 _____
- 6 _____
- 7 _____
- 8 _____
- 9 _____

1 In R, classic multidimensional scaling is implemented through function
2 `cmdscale()`.

3 Note in the example below how the `dist()` function is used in conjunc-
4 tion with `cmdscale()`. The argument `k` controls how many dimensions
5 should be returned.

```
> mdsout = cmdscale(  
+   dist(YCshifts[complete.cases(YCshifts),]), k=2)
```

6 In this example, each “observation” is a single shift in the yield curve. We
7 calculate the distances between all pairs of yield curves, and then seek
8 the two-dimensional mapping that preserves these distances to the extent
9 possible.

1 **Exercise:** Compare the results from multidimensional scaling to those
2 found using PCA. What conclusion can you draw?

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

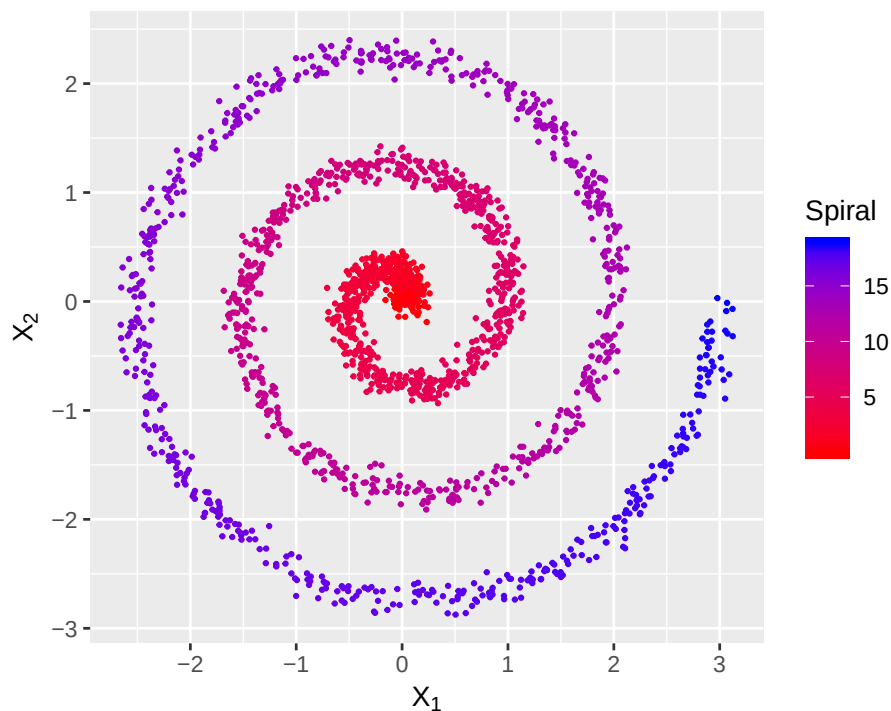
- 1 The criterion minimized by MDS can be written as follows:

2
$$\text{Stress}(x_1, x_2, \dots, x_n) = \left(\sum_{i \neq j} (d_{ij} - \Delta(x_i, x_j))^2 \right)^{1/2}$$

- 3 where d_{ij} is the “true” distance between observations i and j , and $\Delta(x_i, x_j)$
4 is the distance in the lower-dimensional mapping.

- 5 Although in “classical” MDS the distances utilized are Euclidean, the gen-
6 eral concept of “metric” MDS can use any distance d_{ij} which is a properly
7 defined distance metric.

- 1 Consider a synthetic data set where points lie on a spiral:



1 **Exercise:** Does Euclidean distance provide a meaningful measure of “dis-
2 similarity” for the data depicted in the spiral example?

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

1 Some approaches to dimension reduction attempt to construct an im-
2 proved measure of “distance” or “dissimilarity” and incorporate that
3 into metric MDS. The above spiral example is a (classic) illustration of
4 why that can be necessary. (Imagine drawing a new pair of axes onto
5 the spiral example. Can you find any axis system that will yield a useful
6 representation of the data?)

7 Consider the following observation: Although Euclidean distance is not
8 a useful **global** measure of dissimilarity, it is useful on **local** scales. In
9 other words, Euclidean distance could be used to identify two observa-
10 tions which are very similar.

1 ISOMAP

2 **Isomap** is an approach to nonlinear dimension reduction built on MDS
3 with a well-chosen **geodesic distance** between points. The steps:

4 1. **Connect all “neighboring” observations.** This can be achieved by
5 finding all observations within distance ϵ or by find the K nearest
6 neighbors of each observation.

7 2. **Assign weights to connected observations.** The weight is equal to the
8 Euclidean distance separating them.

9 3. **Calculate the geodesic distance between all pairs of points.** The
10 shortest path between points using the provided weights.

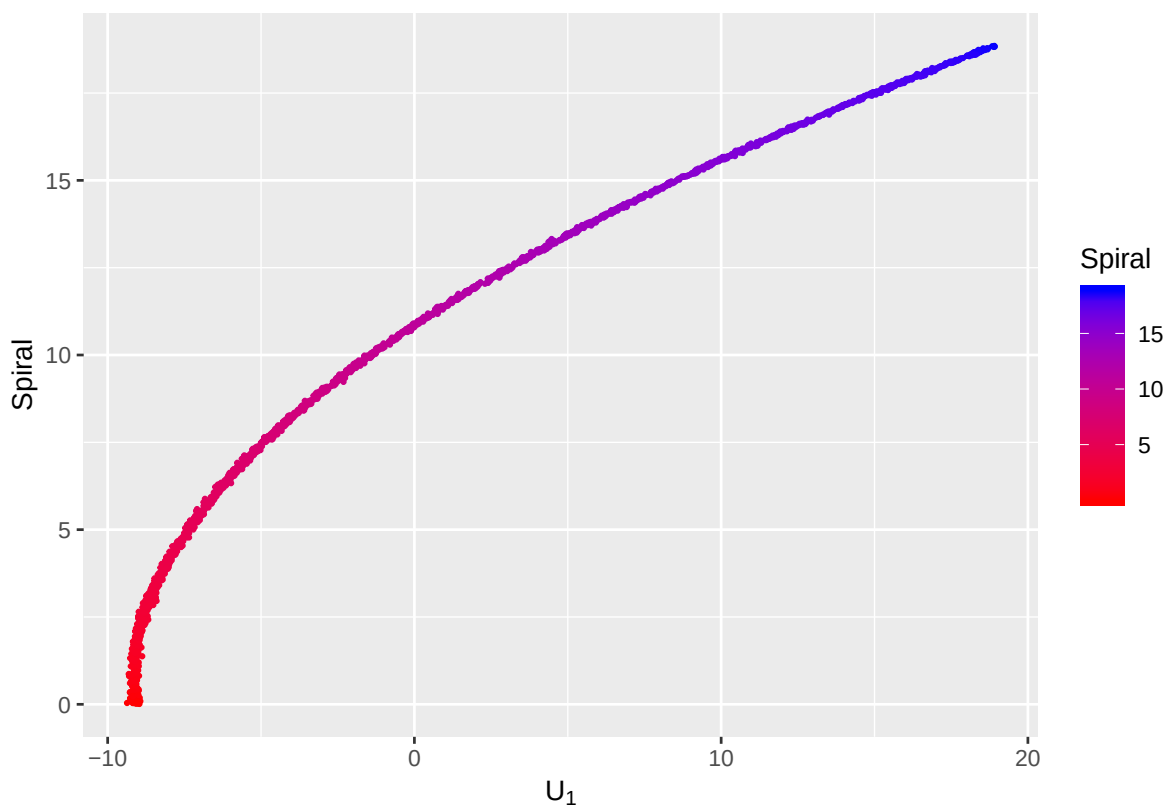
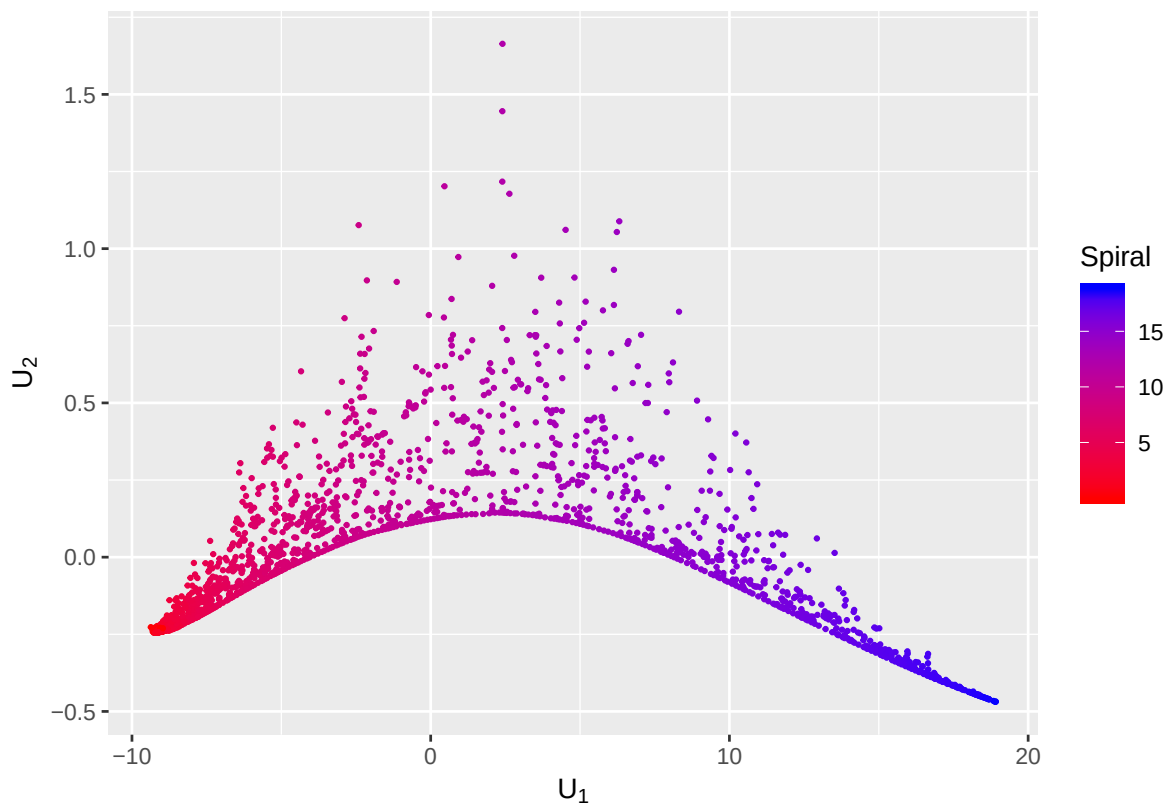
11 4. **Use MDS on the resulting distance matrix.**

1 The following slides show the result of applying the Isomap algorithm to
2 the spiral example.

3 In this case $K = 10$ is utilized. Note that choosing K too small creates the
4 risk of having a “fragmented” graph in which there are observations with
5 no connecting path.

6 The first graph shows the new two-dimensional representation created by
7 Isomap. The color indicates the position along the spiral. We observe that
8 U_1 maps well to the spiral position.

9 This is confirmed by the following plot. Note that there is no reason to
10 expect or prefer that there be a linear relationship between U_1 and the spiral
11 position. The strictly monotonic relationship is sufficient.



1 Stock Example

2 In this example we will consider a sample of 1000 randomly chosen NYSE
3 stocks. For each, a time series of their closing price is obtained, covering
4 30 trading days from August 18, 2017 to September 29, 2017.

5 Read in the data:

```
> stocksample = read.table("stocksample.txt", header=T,  
+                          sep="\t", comment.char="")
```

6 Scale each time series to have mean zero and variance one, so that they can
7 be fairly compared.

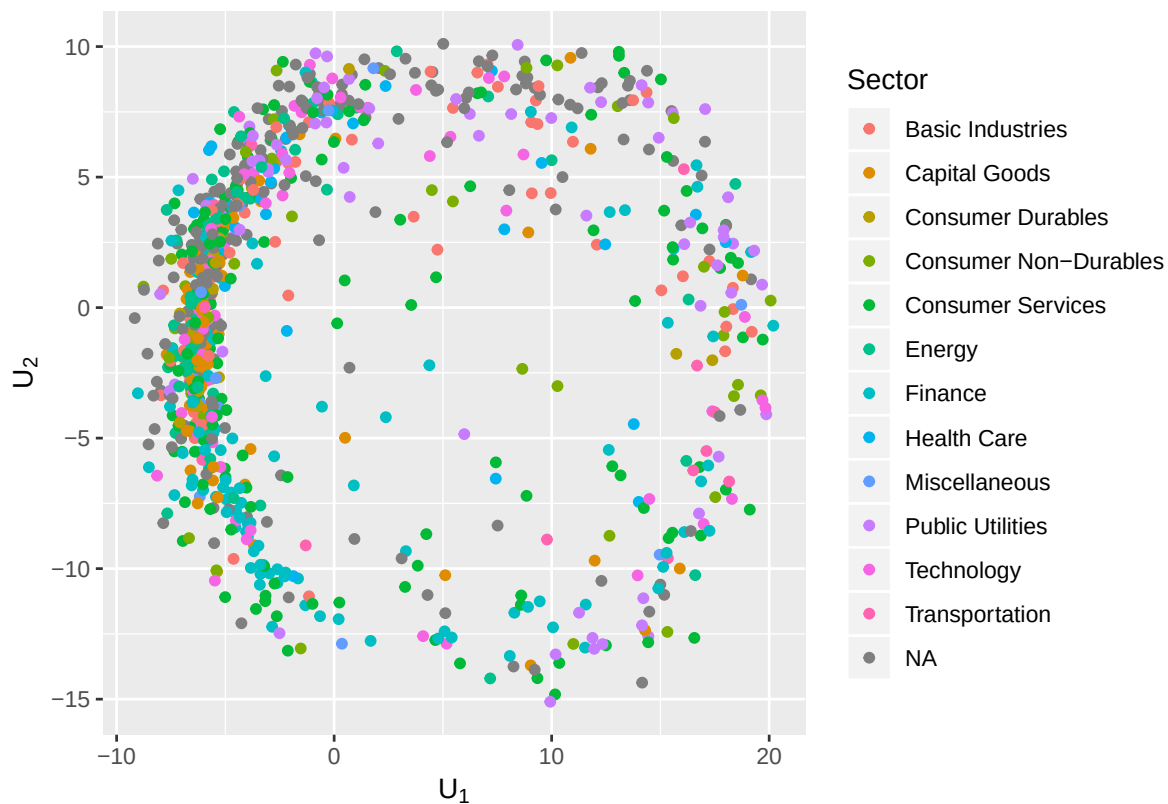
```
> stocksamplescl = apply(stocksample[, 5:34], 1, scale)
```

1 Construct a distance matrix between the time series, and then run the
2 Isomap procedure with $k = 5$:

```
> stockdistmat = dist(t(stocksamplescl))  
> isooutstocks = isomap(stockdistmat, k=5)  
> stocksample = cbind(stocksample, isooutstocks$points[, 1:5])
```

3 Now create a plot looking at the first two dimensions, coloring the points
4 by sector:

```
> ggplot(stocksample, aes(x=stocksample[, 35], y=stocksample[, 36],  
+                          color=sector)) + geom_point() +  
+   labs(x=expression(U[1]), y=expression(U[2]), color="Sector")
```



- 1 **Exercise:** Consider the following R commands. Use this to explore the
 2 structure found in this low-dimensional representation. Can you find a
 3 relationship between position in this two-dimensional space and shape of
 4 the time series?

```
> plot(stocksampl[, 35], stocksampl[, 36], pch=16)
> identify(stocksampl[, 35], stocksampl[, 36])
```

- 5 _____
- 6 _____
- 7 _____
- 8 _____
- 9 _____
- 10 _____

1 Diffusion Map

2 A criticism of Isomap is that it is computationally intensive to calculate the
3 geodesic distances. Also, Isomap is not as robust to noise in the data as are
4 some other approaches, including the one described next, **diffusion map**.

5 This approach is again built on MDS, but the underlying distance between
6 pairs of points is calculated from **random walks** constructed on the ob-
7 served data. Steps in this random walk are taken from observation to ob-
8 servation based on proximity in Euclidean distance (typically), but only
9 neighbors at short distances are candidates for stepping to.

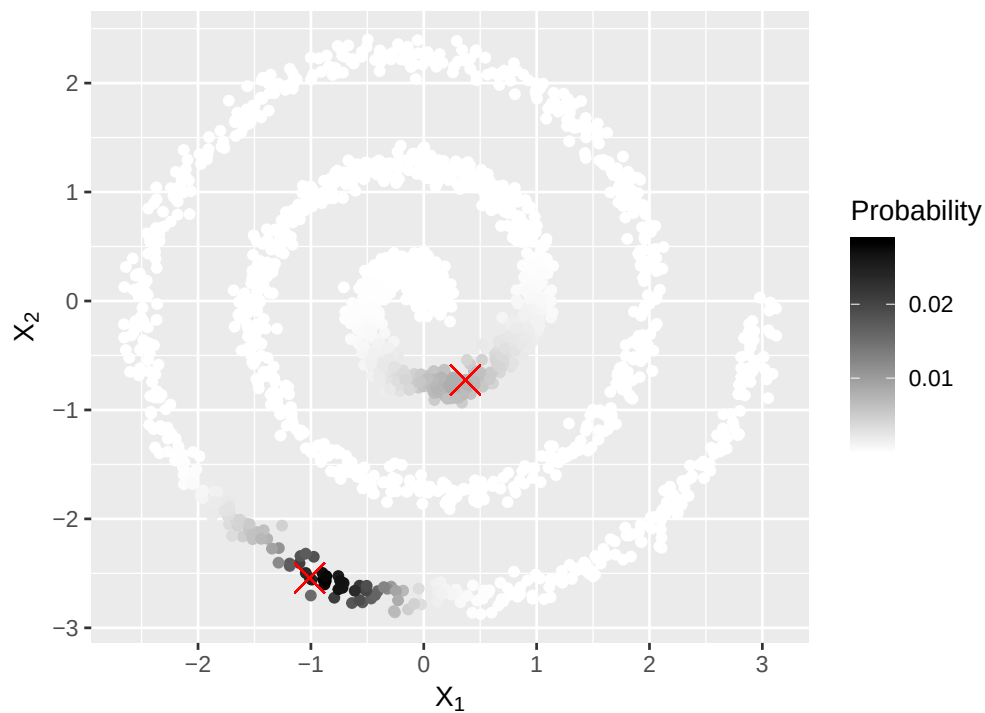
10 If the random walk “diffuses” over enough steps, the result is a **distri-**
11 **bution** over the observations. Comparing distributions between different
12 starting points gives a natural measure of global proximity.

1 The figure on the next slide illustrates the idea.

2 Two observations in the data set are marked with “X.” For either X,
3 imagine a random walk that takes ten steps away from that observation.
4 Steps are small, i.e., the probability of stepping to an observation decreases
5 rapidly as a function of its distance from the current position in the walk.

6 The ending position of the random walk is, of course, random. The shad-
7 ing shows the probability that the random walk ends at various observa-
8 tions.

9 The dissimilarity of these distributions is a natural way of quantifying the
10 distance between them.



Technical Details on Diffusion Map

The process starts by constructing a **similarity measure** between pairs of observations. There is some flexibility in how this is done, but a standard approach is quantify the similarity between observations i and j as

$$s_{ij} = \exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2/\epsilon)$$

where ϵ is a tuning parameter.

Next, the probability of stepping from observation i to any other observation j is

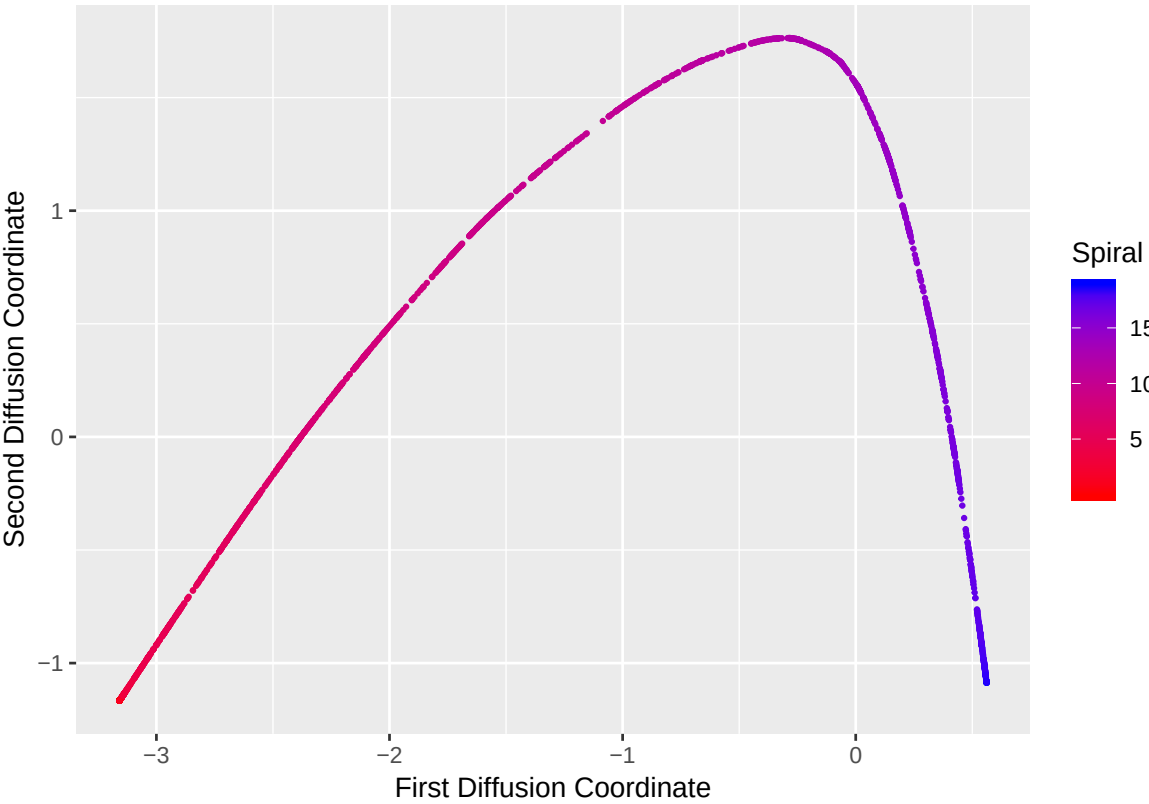
$$p_{ij} = s_{ij} / \sum_k s_{ik}$$

Note that this scaling will force $\sum_j p_{ij} = 1$.

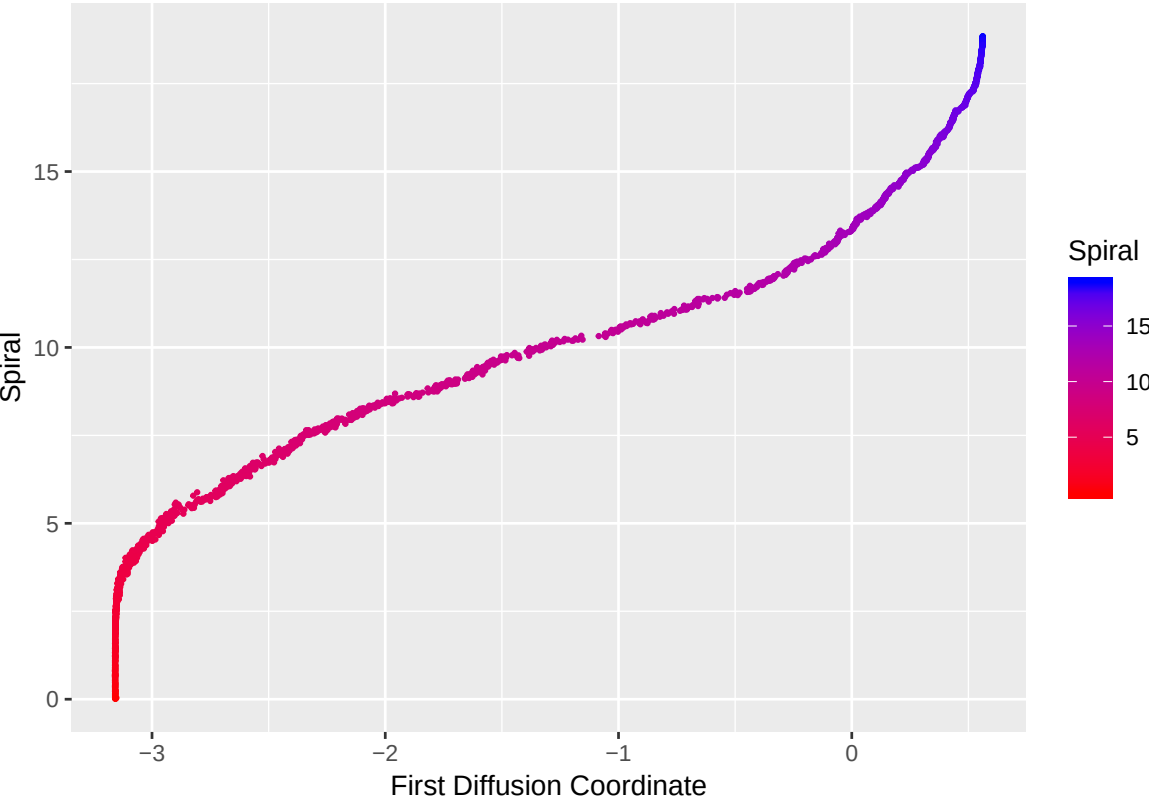
This completely defines the behaviour of the random walk.

- 1 The choice of ϵ is crucial. If it is chosen too small, then steps of the random
- 2 walk will likely move only to other observations which are very similar,
- 3 and, in particular, remain at the same observation. (A “step” can be to the
- 4 same observation.)
- 5 If ϵ is chosen too large, then it will be possible to step to observations which
- 6 are actually quite dissimilar, and the result will be a mapping that does not
- 7 adhere to the lower dimensional structure of interest.
- 8 There is no “right” choice of ϵ . Ultimately, different values should be tried,
- 9 and the chosen value should be that which yields a useful low-dimensional
- 10 representation.

- 1 The slides on the next page show the results when running diffusion map
- 2 on the spiral example.
- 3 In this case, $\epsilon = 0.05$, and 10-step random walks are utilized.
- 4 The random walk is much more robust to noise in the data than is the
- 5 geodesic distance. One way to see this is to note that small perturbations in
- 6 the position of any observation will have very little effect on the diffusion
- 7 distances, but could potentially affect geodesic distances a great deal.
- 8 Also, there are computational shortcuts that make calculating the diffusion
- 9 map quick. Much of this is based on the theory of **Markov chains**.



1



1

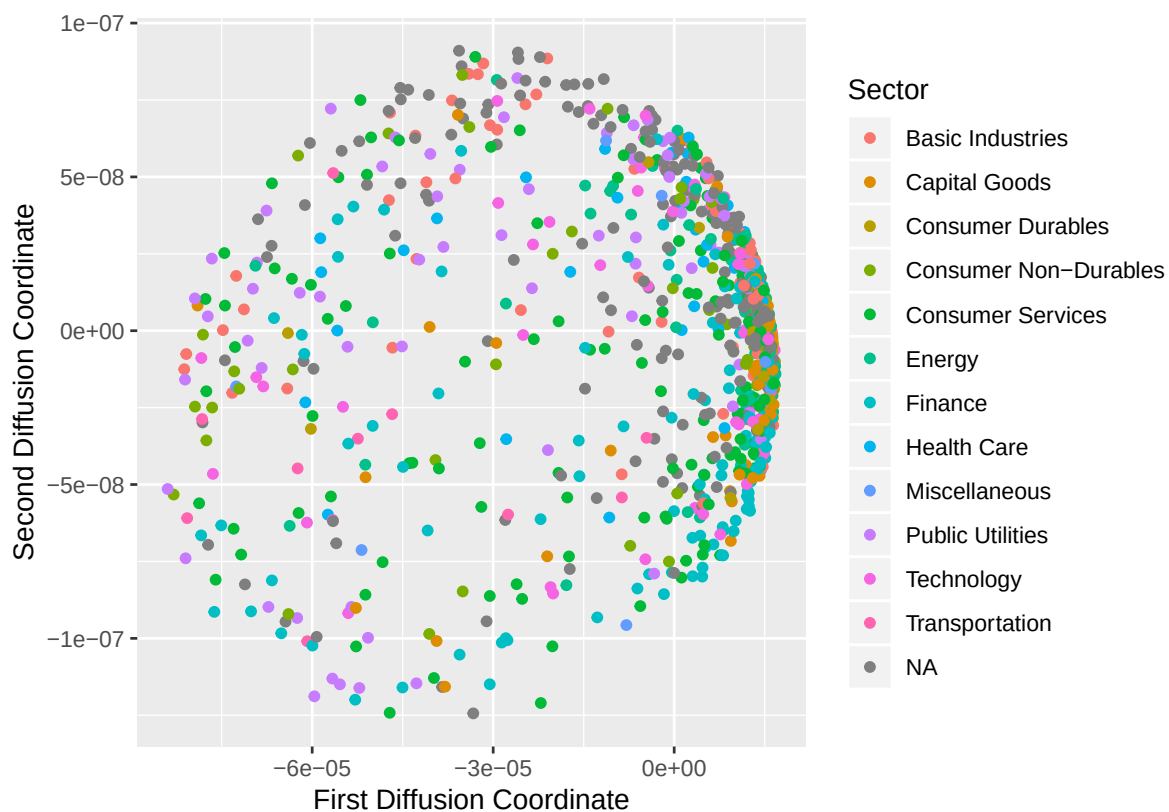
1 Return to the Stock Example

- 2 Diffusion map is implemented in the package `diffusionMap` via the
3 function `diffuse()`. The arguments `eps.val` and `t` set the values of ϵ
4 and the length of the random walk, respectively.

```
> stockdiffmap = diffuse(stockdistmat, eps.val=50, t=10)
```

- 5 The component `X` of the output contains the coordinates in diffusion space.

```
> stocksample$dmap1 = stockdiffmap$X[,1]  
> stocksample$dmap2 = stockdiffmap$X[,2]
```



- 1 **Exercise:** Again, explore the space to see how the shape of the time series
2 varies over diffusion space.

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

1 The Options Example

- 2 Let's now revisit the options sample previously described, and look at the
3 four quantities that we would consider as “predictors” in a model for the
4 price.

```
> optionsdatasub = optionsdata[, c(2, 3, 9, 10)]
```

- 5 To construct a distance metric in this case, we need to adjust for the fact that
6 the four variables are not comparable. Here, we introduce the **Mahalanobis**
7 **Distance** as a useful approach. The distance between two vectors $\mathbf{x}_1$ and
8 $\mathbf{x}_2$ is

$$9 \quad (\mathbf{x}_i - \mathbf{x}_j)^T \widehat{\Sigma}^{-1} (\mathbf{x}_i - \mathbf{x}_j)$$

- 10 where $\widehat{\Sigma}$ is the sample covariance matrix of the $\mathbf{x}$'s.

- 1 **Exercise:** Explain why the weighting by the inverse covariance matrix is a
2 good approach.

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

- 1 We can calculate the Mahalanobis distances as follows, using the function
2 `distances()` from the package of the same name:

```
> optdist = distance_matrix(distances(optionsdatasub,  
+                             normalize="mahalanobize"))
```

- 3 Note that the function `distance_matrix()` extracts the distance matrix
4 from the output of `distances()`.

- 5 Now we use `Isomap` and store the mappings:

```
> optiso = isomap(optdist,k=10)  
> optionsdata$iso1 = optiso$points[,1]  
> optionsdata$iso2 = optiso$points[,2]  
> optionsdata$iso3 = optiso$points[,3]
```

- 1 **Exercise:** Construct plots to compare the Isomap-constructed representa-
2 tion with the four input quantities, and also with the ask price. Be sure to
3 look at the following plot:

```
> ggplot(optionsdata, aes(x=iso1,y=iso3,color=log10(ask))) +  
+   geom_point(size=1) +  
+   scale_color_continuous(low="blue",high="red") +  
+   labs(x=expression(U[1]), y=expression(U[2]), color="Log Ask")
```

- 4 _____
5 _____
6 _____
7 _____
8 _____
9